

✓ AI활용프로그래밍 Week 1 실습 노트북

프로그래밍 언어 구조와 LLM (With AI 코딩 시작하기)

이 노트북은 **Week 1 강의자료** 내용을 바탕으로, 대학생(초보)도 **바로 실행하며 따라할 수 있도록** 구성했습니다.

오늘 학습 목표

1. 문법(Syntax)과 의미/논리(Semantics/Logic)의 차이를 설명할 수 있다.
2. 오류 3종류(Syntax/Runtime/Logic)를 구분할 수 있다.
3. 디버깅 기본 루틴을 적용할 수 있다.
4. LLM(ChatGPT)을 코딩 학습에 활용하는 기본 프롬프트 템플릿을 실습한다.

사용 팁: 각 셀을 클릭하고 **Shift + Enter** 로 실행하세요.

✓ 0. 준비: 출력과 계산이 잘 되는지 확인

```
print("Hello, Jupyter!")
```

✓ 1. 문법(Syntax) vs 의미/논리(Semantics/Logic)

- **문법 오류(Syntax Error)**: 규칙을 어겨서 **실행 자체가 불가능**
- **논리 오류(Logic Error)**: 실행은 되지만 **의도와 다른 결과**

✓ 1-1. 문법 오류(Syntax Error) 실습

아래 코드는 일부러 **따옴표/괄호를 닫지 않은** 예시입니다.
실행하면 어떤 메시지가 뜨는지 확인해보세요.

⚠️ 주의: 이 셀은 에러를 내기 위한 셀입니다.

```
# 문법 오류 예시 (일부러 에러)  
print("Hello"
```

✓ 1-2. 논리 오류(Logic Error) 실습

아래 코드는 실행은 되지만 평균 계산이 **틀리게** 되어 있습니다.
왜 틀렸는지, 어떻게 고칠지 생각해보세요.

```
scores = [80, 90, 100]  
avg = sum(scores) / 2 # <- 의도는 3으로 나누기!  
print("평균:", avg)
```

✅ 수정: `len(scores)`로 나누도록 고쳐봅시다.

```
avg = sum(scores) / len(scores)  
print("수정된 평균:", avg)
```

✓ 2. 오류의 3종류

1. **Syntax Error**: 문법 규칙 위반 (대부분 실행 전 발견)
2. **Runtime Error (Exception)**: 실행 중 예외 발생
3. **Logic Error**: 실행은 되지만 결과가 틀림

2-1. Runtime Error(런타임 오류) 실습: 인덱스 범위 오류

리스트 `nums`의 길이는 3이므로 인덱스는 `0,1,2`까지만 접근 가능합니다.
아래 코드를 실행해보고, 에러 메시지를 확인하세요.

⚠ 주의: 이 셀은 에러를 내기 위한 셀입니다.

```
nums = [1, 2, 3]
print(nums[3]) # 존재하지 않는 인덱스 접근
```

✅ 해결 방법(예시)

- 마지막 원소는 `nums[2]` 또는 `nums[-1]` 사용
- 길이를 먼저 확인: `len(nums)`

```
nums = [1, 2, 3]
print("len(nums) =", len(nums))
print("마지막 원소(nums[-1]) =", nums[-1])
```

3. 디버깅 기본 루틴(중요)

디버깅은 감으로 고치는 게 아니라 **절차**가 있습니다.

1. **증상 재현**: 입력/환경을 고정하고 언제 오류가 나는지 확인
2. **Traceback 읽기**: 에러 종류(Error Type) + 줄 번호(Line)부터 보기
3. **최소 재현 코드(MRE)** 만들기: 문제를 가장 작은 형태로 줄이기
4. **가설 → 한 번에 한 가지 변경 → 재실행**
5. 수정 후 **정상 케이스 + 엣지 케이스** 테스트

아래는 “작게 실행하고 자주 확인”하는 연습입니다.

3-1. 작은 단위로 실행하기: 합계 구하기

```
total = 0
for i in range(1, 6):
    total += i
print("합계:", total)
```

확인 질문

- `range(1, 6)`은 어떤 숫자들을 만들까요?
- 결과가 15가 되는 이유를 말로 설명해보세요.

3-2. 같은 기능을 더 간단히: sum() 사용

```
print("sum(range(1,6)) =", sum(range(1, 6)))
```

4. With AI 코딩(LLM 활용) 실습

LLM은 **정답 제조기**가 아니라 **초안 생성 + 코치**에 가깝습니다.
따라서 항상 **실행/검증**이 필요합니다.

프롬프트 템플릿(복사해서 ChatGPT에 붙여넣기)

아래 템플릿에서 코드 부분만 바꿔서 사용하세요.

역할: 너는 친절한 파이썬 튜터야.
목표: 아래 코드를 초보가 이해하도록 설명해줘.
맥락: 나는 1학년이고 파이썬을 막 시작했어.

제약: 어려운 용어는 최소화하고, 핵심을 5줄로 요약해줘.

출력 형식: (1) 한 줄 요약 (2) 줄별 설명 (3) 실수하기 쉬운 포인트 1개

코드:

```
```python
여기에 코드 붙여넣기
```

#### 4-1. 실습 1: 아래 코드 설명을 ChatGPT에 요청해보기

```
total = 0
for i in range(1, 6):
 total += i
print(total)
```

✅ 과제(노트북 안에서 기록하기)

- ChatGPT 답변을 읽고, **내가 이해한 설명**을 아래 셀에 3줄로 적어보세요.

(여기에 본인 설명을 작성하세요)

#### 5. 리팩터링(가독성 개선) 실습

리팩터링은 **동작은 그대로** 두고, **읽기 쉽게** 바꾸는 작업입니다.

##### 5-1. 리팩터링 대상 코드

```
a = 10; b = 20
c = a + b
print("sum=", c)
```

##### 5-2. 실습 미션

1. 변수 이름을 더 의미 있게 바꾸기
2. 주석 2줄 추가
3. 출력은 동일하게 유지

✅ 아래 셀에 본인 리팩터링 코드를 작성해보세요.

# TODO: 여기에 리팩터링한 코드를 작성하세요.

##### 5-3. 예시 답안(비교용)

```
num1 = 10
num2 = 20

두 숫자를 더합니다.
total_sum = num1 + num2

print("sum=", total_sum)
```

#### 6. AI 답변 검증 체크리스트

AI가 준 코드/설명은 반드시 검증합니다.

- ☐ 코드를 직접 실행했는가?
- ☐ 요구사항/제약을 만족하는가?

- ☐ 엣지 케이스(빈값, 0, 큰 값)를 생각했는가?
- ☐ 필요한 라이브러리/버전이 맞는가?
- ☐ 내가 “왜 이렇게 동작하는지” 설명할 수 있는가?

## 7. 미니 퀴즈(5문항) — 노트북에서 답 적기

아래 질문에 대해 정답(예: 1-B)을 작성해보세요.

- 문법 오류(Syntax Error)는 보통 언제 발견될까? A. 실행 중 B. 실행 전 C. 테스트 후
- LLM 답변에 대한 올바른 태도는? A. 무조건 믿는다 B. 참고하고 검증한다 C. 읽지 않는다
- 디버깅의 첫 단계로 가장 좋은 것은? A. 랜덤 수정 B. 증상 재현 C. 코드 전체 삭제
- 좋은 프롬프트에 포함되면 좋은 요소가 아닌 것은? A. 목표 B. 출력 형식 C. 개인정보
- 논리 오류(Logic Error)의 특징은? A. 실행 불가 B. 실행 가능하지만 결과가 틀림 C. 컴파일이 느림

✅ 정답을 여기에 작성하세요:

- 1)
- 2)
- 3)
- 4)
- 5.

## 8. 과제 안내

### 프롬프트 3종 작성

(1) 설명 요청 / (2) 디버깅 요청 / (3) 리팩터링 요청

각 프롬프트에 대해

- 프롬프트 원문
- AI 답변 요약
- 내가 검증/수정한 점
- 배운 점

을 함께 정리해서 제출하세요.

목표는 “AI를 써서 더 빨리”가 아니라 “AI와 함께 더 정확히”입니다.

### 8-1. 과제 작성 공간(복사해서 3개 작성)

#### (1) 설명 요청 프롬프트

- 프롬프트:
- AI 답변 요약:
- 내가 수정/검증한 점:
- 배운 점:

#### (2) 디버깅 요청 프롬프트

- 프롬프트:
- AI 답변 요약:
- 내가 수정/검증한 점:
- 배운 점:

#### (3) 리팩터링 요청 프롬프트

- 프롬프트:
- AI 답변 요약:
- 내가 수정/검증한 점:
- 배운 점:

## 9. 오늘 정리

- 프로그래밍 언어는 **문법·의미·실행** 구조로 이해할 수 있다.
- 오류는 **문법/런타임/논리**로 나뉜다.
- 디버깅은 **재현→분리→검증** 절차가 핵심이다.
- LLM은 강력한 도우미지만 **검증과 기록이 필수**다.

다음 주: **파이썬 개발환경 구축 + Hello World**